

Classifiez automatiquement des biens de consommation

Moteur de classification des articles en différentes catégories, à partir du texte pour l’entreprise ***Place de Marché***



par Dimitri

OPENCLASSROOMS

Made with GAMMA



Contexte du projet

Objectif principal

Automatiser l’attribution de la catégorie d’un produit sur une marketplace à partir de sa description (texte) et de sa photo, afin de fiabiliser la catégorisation et de passer à l’échelle.

Sources de données

Jeu de données produits (descriptions en anglais + images) : 1050 articles répartis en 7 catégories équilibrées.

Enjeu métier

Réduire la catégorisation manuelle et améliorer l’expérience vendeur/acheteur : mise en ligne plus rapide, recherche plus pertinente et meilleure qualité de catalogue.

Made with GAMMA

Sommaire

1. Introduction
Contexte : marketplace e-commerce, catégorisation manuelle peu fiable
Objectif : évaluer la faisabilité d'une classification automatique par catégorie
Données : descriptions (anglais) + images produits

2. Mise en place de l'environnement
Outils : Python, Pandas, scikit-learn, TensorFlow/Keras, NLP (Sentence Transformers / TF Hub)
Chargement du dataset et structuration des chemins d'images

3. Exploration et qualité des données
Contrôles : valeurs manquantes, doublons, cohérence des labels
Analyse des descriptions (longueurs, nettoyage)
Vérification disponibilité/format des images

4. Étude de faisabilité - Approche Texte
Prétraitement texte
Extraction de features : Count, TF-IDF, Word2Vec, USE, SBERT
Visualisation 2D (PCA→t-SNE) + clustering KMeans
Évaluation : ARI / NMI

5. Étude de faisabilité - Approche image
Baseline : ORB-BoVW
Transfer Learning : features ResNet50
Visualisation 2D + clustering KMeans
Évaluation : ARI / NMI + conclusion comparative

6. Classification supervisée à partir des images
Modèle Transfer Learning (MobileNetV2) + data augmentation
Résultats : accuracy, classification report, matrice de confusion
Analyse des erreurs et limites

7. Collecte de données – API OpenFoodFacts
Extraction de 10 produits "champagne"
Génération d'un CSV + récupération d'images
Objectif : ouverture vers l'épicerie fine

8. Conclusion et recommandations
Synthèse des résultats (texte vs image)
Recommandation : approche image-first, pistes d'amélioration (fusion multimodale, + données)

Made with GAMMA

Problématique et objectifs

Problématique

La marketplace “Place de marché” souhaite automatiser l’attribution des catégories produits.
Aujourd’hui, la catégorisation est réalisée manuellement par les vendeurs : elle est lente, hétérogène et source d’erreurs, ce qui dégrade la recherche produit et la qualité du catalogue.
L’enjeu est de vérifier s’il est possible de regrouper et classer automatiquement les produits à partir de deux sources d’information disponibles à grande échelle : la description textuelle (anglais) et l’image produit.

Objectifs

- Préparer et contrôler la qualité des données (texte + images + labels).
- Tester plusieurs représentations texte (Count, TF-IDF, Word2Vec, USE, SBERT) et mesurer la capacité de regroupement (t-SNE, KMeans, ARI/NMI)
- Tester plusieurs représentations image (ORB-BoVW, Transfer Learning CNN) et mesurer la capacité de regroupement (t-SNE, KMeans, ARI/NMI).
- Comparer objectivement texte vs image et conclure sur la faisabilité.
- Mettre en place une classification supervisée à partir des images (Transfer Learning + data augmentation) et analyser les erreurs.

Made with GAMMA

Mise en place de l’environnement

Début du projet

Approfondissement de python

Installation de python et de ses librairies en local

Outils utilisés

Travail sur GoogleCollab

Adaptation sur Notebook pour le livrable

Rythme de travail

Travail de un jour par semaine (Alternance) + Prise de temps pour découverte de python, cours en libre service etc…



Jeu de données

Le dataset produits contient les informations suivantes :

1

Descriptions textuelles

Description produit en anglais (champ “description”) + éventuellement nom du produit.

Sert à tester des méthodes pour regrouper automatiquement les produits.

2

Images produits

Photo principale du produit (fichier image + chemin d’accès).

Sert à extraire des caractéristiques visuelles.

3

Labels et métadonnées

Catégorie réelle (7 classes) utilisée comme vérité terrain pour évaluer les regroupements.

Données d’identification produit (id/URL) pour relier texte, image et catégorie.





Librairie Python utilisées

1 Analyse et traitement des données

pandas *: chargement du dataset, sélection des colonnes utiles, contrôles qualité*

numpy *: opérations numériques, calculs rapides (statistiques, transformations)*

2 Visualisation

matplotlib *: graphiques (t-SNE, PCA, courbes d'entraînement)*

PIL / opencv-python*: lecture, redimensionnement, affichage d'images et extraction ORB*

3 Statistiques, ML et DeepLearning

scikit-learn *: vectorisation texte, clustering, réduction de dimension, métrique*

tensorflow / keras *: transfer learning, entraînement supervisé, data augmentation*

sentence-transformers *: embeddings SBERT / USE*

Made with GAMMA

Affichage des données : Les fonctions de bases (Dataframe)



"Un DataFrame est une structure de données en tableau, similaire à une feuille Excel, qui permet de manipuler et analyser des données efficacement."

```
print(df.head())
```

→ Permet d'afficher les 5 premières lignes du tableau pour vérifier que les données sont bien lues et structurées.

```
print(df.info())
```

→ Affichage des métadonnées sur le DataFrame comme le nombre de lignes et de colonnes, le type de données (int, object, float et le nombre de valeurs manquantes

```
print(df.describe())
```

→ Fournit des statistiques pour les colonnes numériques comme la moyenne, la médiane etc...

Made with GAMMA

Traitement des données : Qualité des entrées (texte + images)

Pourquoi il y a des “données problématiques” ?

1

Descriptions hétérogènes

- texte bruité (ponctuation, casse, caractères spéciaux)
- longueur très variable (descriptions très courtes vs très longues)

2

Champs parfois imcomplets

- certains champs comme label ou foodContentsLabel (OpenFoodFacts) peuvent être vides

3

Contraintes images

- tailles/résolutions très différentes
- parfois images très grandes

‘Pourquoi est-il important de les traiter ?’

1

Améliorer la qualité des features

- un texte non nettoyé dégrade TF-IDF/embeddings, une image standardisée dégrade le CNN

2

Éiter les erreurs techniques

- chargement/resize/normalisation, stabilité du pipeline

3

Rendre les comparaisons fiables

- même préparation = résultats ARI/NMI comparables entre méthodes

Made with GAMMA

Traitement des données : Les valeurs manquantes

Comment on les a traitées ?

Texte

- nettoyage simple : minuscules, suppressions ponctuation / caractères inutiles
- création d'une colonne "description_clean"
- contrôle des longueurs (avant/après nettoyage)

Images

- contrôle présence des fichiers
- standardisation : redimensionnement
- pipeline stable pour l'extraction

```
Longueur description (brut) :
count    1050.000000
mean      473.820952
std       457.910422
min       109.000000
25%       192.000000
50%       278.000000
75%       588.250000
max      3490.000000
Name: desc_len, dtype: float64

Longueur description (clean) :
count    1050.000000
mean      435.297143
std       432.537688
min        69.000000
25%       170.000000
50%       248.000000
75%       536.750000
max      3368.000000
```

Made with GAMMA

Visualisation des données : projections & évaluation



Matplotlib

génération des figures (t-SNE, PCA→t-SNE, courbes d'entraînement, matrice de confusion)

Pourquoi l'utiliser ?

- Valider visuellement la séparabilité des catégories
- Comparer les approches sur un même format de visualisation
- Diagnostiquer les limites et guider la suite



Seaborn

heatmap propre pour la matrice de confusion / comparaison rapide

Utilisation pour le projet

Projections 2D : PCA -> t-SNE des features (TF-IDF, Word2Vec/USE/SBERT, ResNet50)

Visualisation des clusters, qualité du clustering et performance supervisée

Made with GAMMA

TF-IDF + t-SNE + KMeans

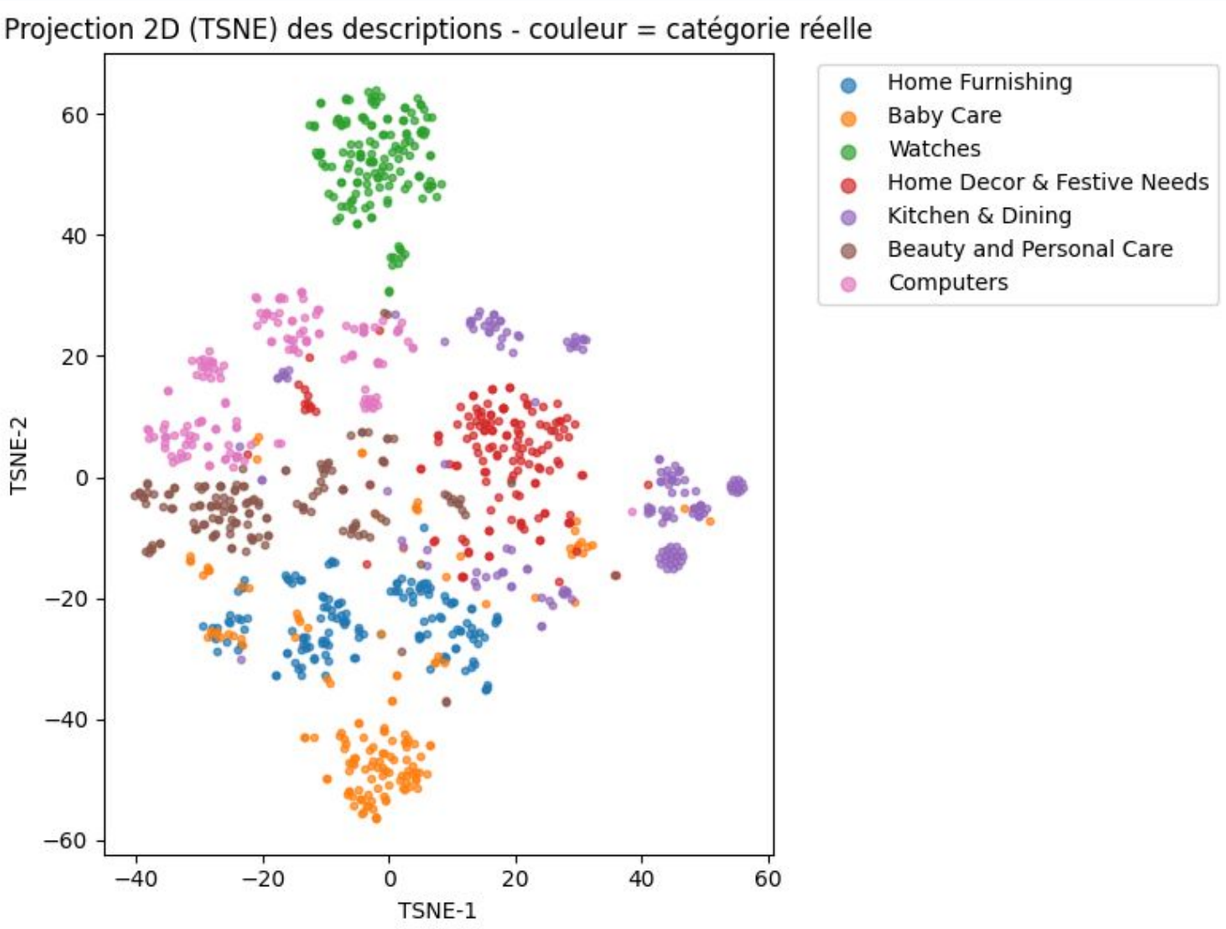
Ce qu'on fait :

On transforme les descriptions texte en vecteurs (TF-IDF), puis on regarde si les produits se regroupent naturellement par catégorie

TF-IDF : (1050, 4979) → 1050 produits, ~5000 mots/termes utiles

ARI = 0.284

NMI = 0.484



Cohérent : ARI ~0.28 et NMI ~0.48 indiquent une séparation partielle.

Concrètement : on voit des zones assez homogènes, mais aussi des mélanges entre catégories proches (ex. déco vs home furnishing vs kitchen).

Conclusion : le texte seul aide, mais ce n'est pas assez "propre" pour un regroupement automatique parfait.

Made with GAMMA

CountVectorizer vs TF-IDF : quel encodage choisir ?

Objectif

Comparer deux façons de transformer les descriptions produits en vecteurs pour tester si le texte seul permet de regrouper automatiquement les produits par catégorie.

CountV : On compte combien de fois chaque mot apparaît

TF-IDF : On pondère les mots : les mots fréquents partout comptent moins, les mots spécifiques comptent plus

Méthode (simple)

- Même dataset : 1050 produits, 7 catégories
- Même algo de clustering : KMeans (k=7)
- Même évaluation : comparaison clusters vs catégories réelles via ARI et NMI

Résultats

CountVectorizer (fréquences brutes)

→ ARI 0.134 | NMI 0.301

TF-IDF (mots pondérés, mots discriminants valorisés)

→ ARI 0.266 | NMI 0.520

Le texte contient un signal, mais pas parfait, il faudra comparer avec l'image.

Made with GAMMA

Contrôle & prétraitement des images

Couverture dataset

Images trouvées : 100% (ratio = 1.000)

Hétérogénéité des formats

- Tailles très variables (ex. 273x497 -> 1257x2049)

Prétraitement pour CNN

Conversion RGB, resize 224x224 et normalisation [0-1]



Made with GAMMA

Clustering d’images via CNN (ResNet50) + KMeans

Objectif

- Tester si les images seules permettend de regrouper automatiquement les produits par catégorie

Méthode

- CNN (réseau neuronal pour images) : extrait des motifs visuels (formes, textures)
- ResNet50 (CNN pré-entraîné 50 couches) : modèle robuste pour extraire des “features”
- ImageNet (base d’images géante, 1000 classes) : sert à pré-entraîner ResNet50
- Transfer learning : on n’entraîne pas, on “récolte” des caractéristiques
- Features (2048) : 1 vecteur par image
- KMeans (algorithme qui regroupe par similarité) : crée 7 groupes sans connaître les labels

“Les images portent un signal visuel fort : le regroupement automatique est nettement meilleur que via le texte”

Pipeline

Prétraitement : RGB + resize 224×224 (+ normalisation ResNet).

Extraction : ResNet50 → 2048 features / image.

Regroupement : KMeans avec 7 clusters (= 7 catégories réelles).

Résultats

Matrice : (1050, 2048)

ARI = 0.493 (0 hasard → 1 parfait)

NMI = 0.587 (0 hasard → 1 parfait)

Made with GAMMA

Visualisation 2D des images (ResNet50 -> PCA -> t-SNE)

Objectif

- Vérifier rapidement si les images seules contiennent assez d'information pour séparer les catégories.
- Confirmer visuellement ce que mesureront ensuite ARI/NMI

Ce qu'on observe ici

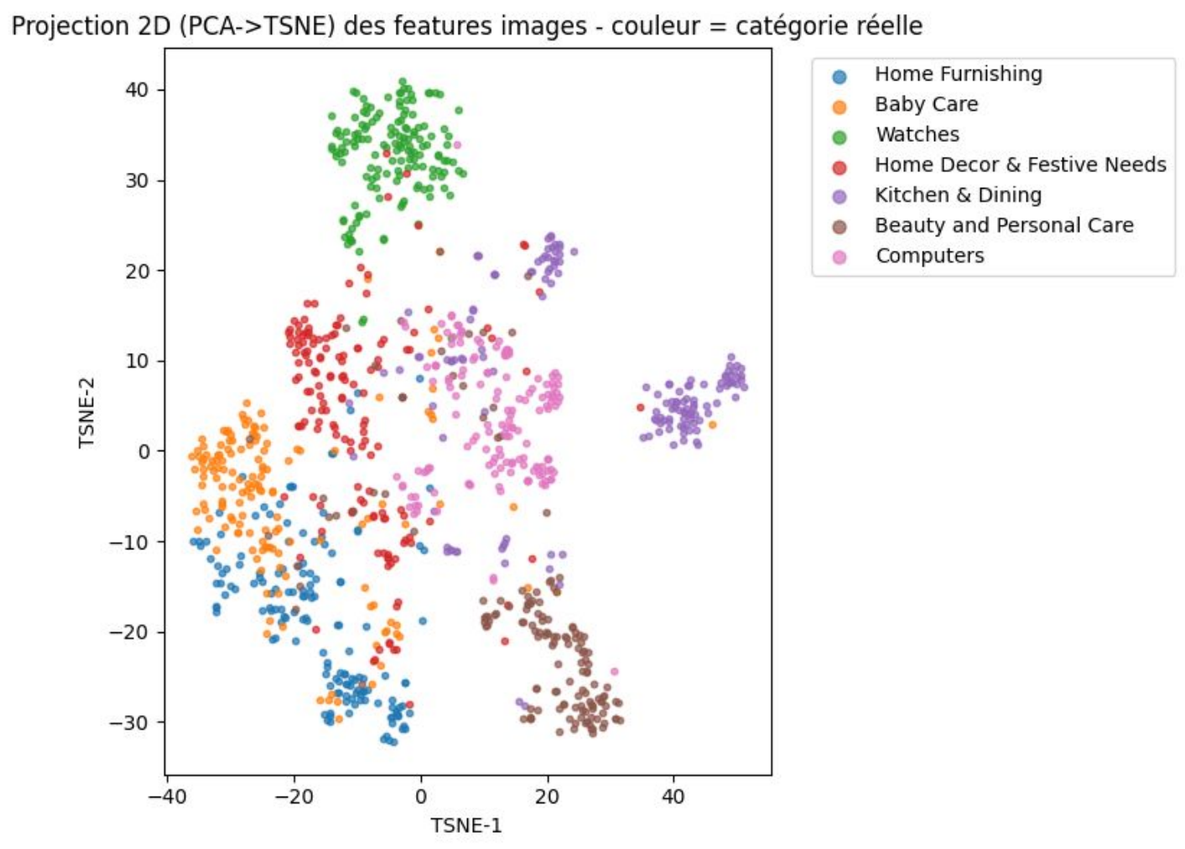
On voit des groupes bien séparés pour certaines catégories

D'autres zones sont plus mélangées

Ce que ça nous apporte

La structure visuelle confirme que la piste "image" est prometteuse.

Ca justifie de passer à l'étape suivante : clustering + métriques, puis classification supervisée



Made with GAMMA

Baseline vision “classique” : ORB + BoVW + KMeans

Pourquoi on le teste ?

- Baseline rapide sans deep learning

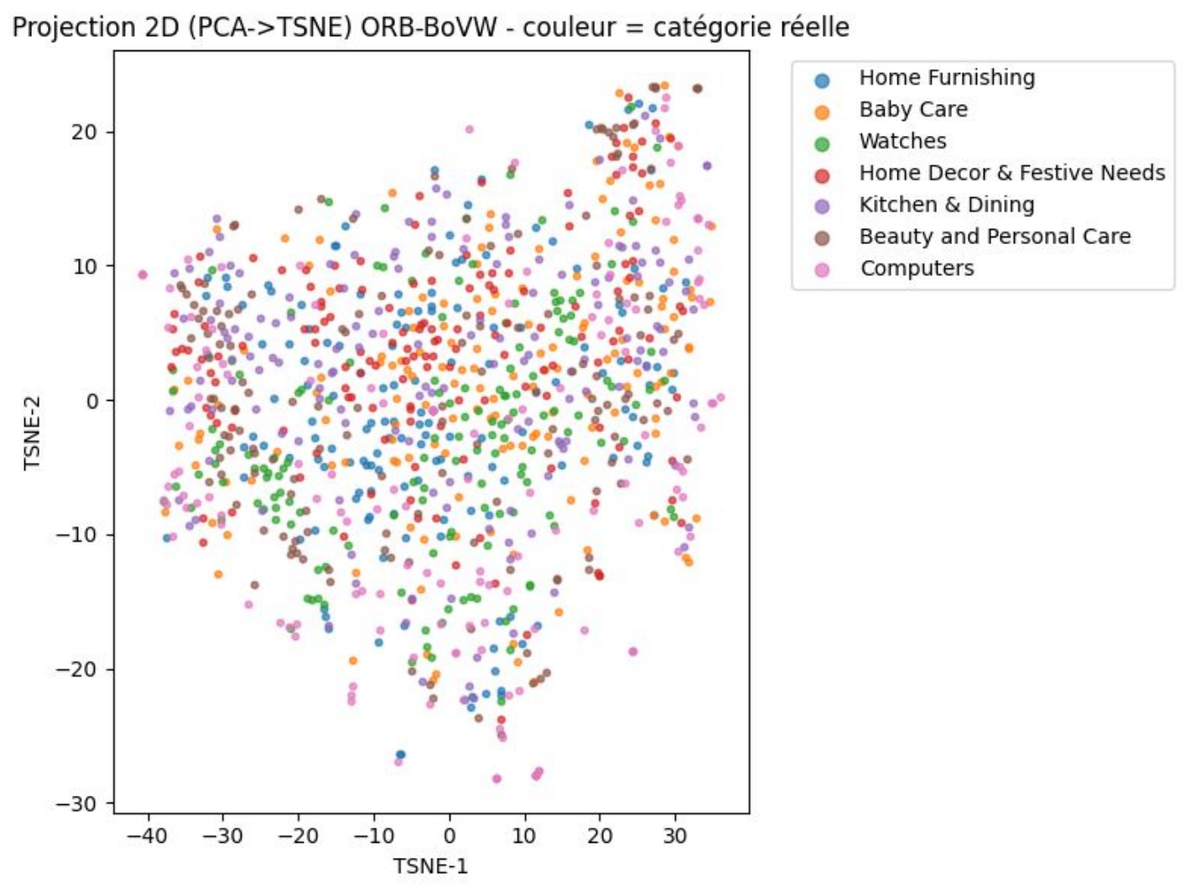
Méthode

ORB (points/texture) -> BoVW (histogramme) -> KMeans (7 clusters)

Résultat clé

ARI : 0.013 - NMI 0.031 = hasard

ORB Capture des textures locales mais pas le type d’objet -> on passe aux CNN



ORB “ne marche pas” ici parce qu’il regarde surtout des petits détails de l’image (motifs, textures) sans vraiment comprendre ce que l’objet représente. Résultat : des produits différents peuvent se ressembler “visuellement” sur ces détails, et le regroupement mélange les catégories.

Made with GAMMA

Synthèse des approches testées (Texte vs Image)

Objectif

- Vérifier si on peut regrouper automatiquement les produits par catégorie

	Approche	ARI	NMI	Commentaire
3	Image - ResNet50 features + KMeans	0.492857	0.587465	Meilleure approche, clusters plus cohérents
1	Texte - TF-IDF + KMeans	0.266000	0.520000	Meilleure que Count, séparation partielle
0	Texte - CountVectorizer + KMeans	0.134000	0.301000	Baseline texte, séparation faible/modérée
2	Image - ORB BoVW + KMeans	0.012588	0.030569	Quasi aléatoire, catégories mélangées

Ce qu'on compare

- Texte (CountV) : compte les mots "brut"
- Texte (TF-IDF) : met plus de poids sur les mots vraiment discriminants
- Image (ORB) : compare des détails visuels (textures/coins)
- Image (CNN ReNet50) : extrait des caractéristiques visuelles "hauts niveaux" (formes, style, objet)

Lecture des scores

ARI / NMI : 0 = Hasard, 1 = Parfait

"Sur ce dataset, l'image est plus fiable que le texte pour retrouver les catégories. Le meilleur compromis en non-supervisé, c'est ResNet50 + KMeans. Le texte aide un peu (TF-IDF > Count), mais reste moins net. ORB ne capte pas le bon signal."

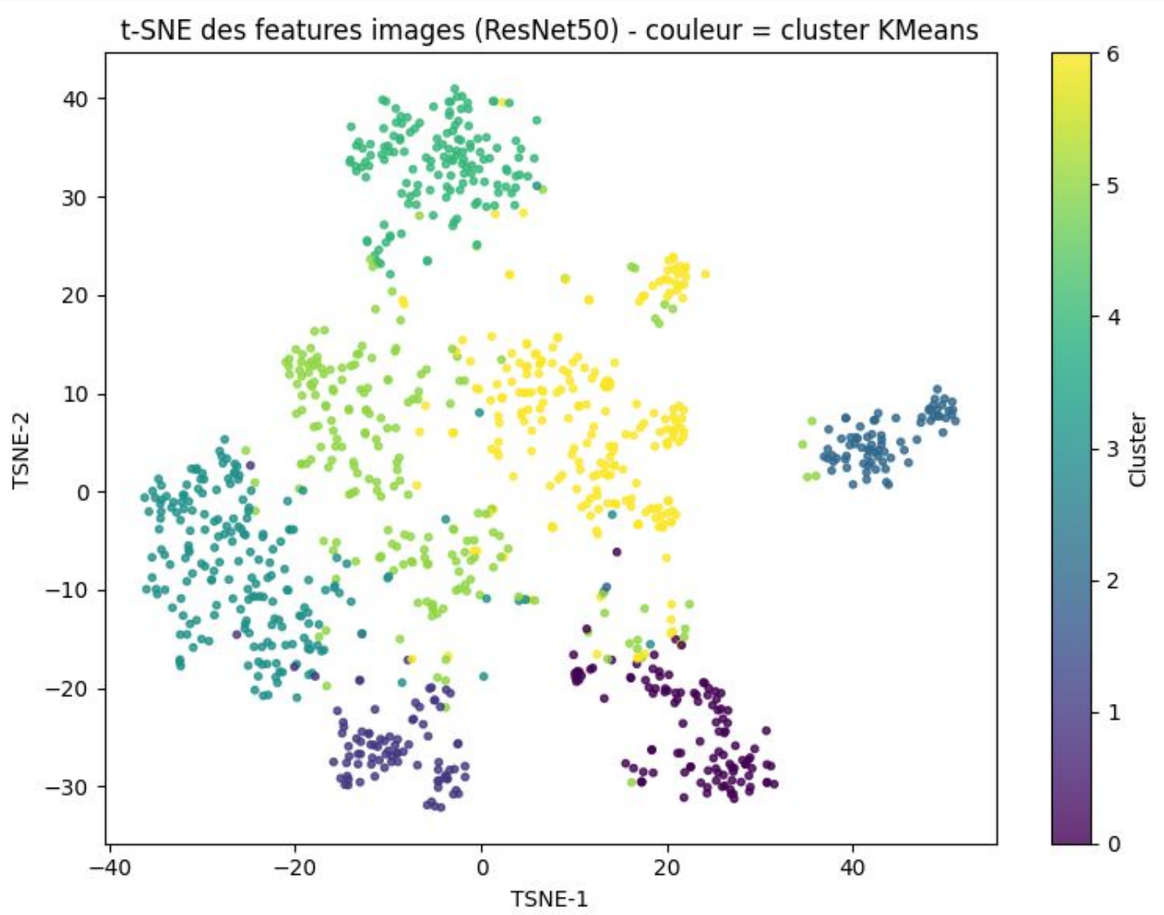
Made with GAMMA

Ce que l’IA regroupe automatiquement dans les images

Pourquoi cette visualisation est utile ?

- On voit que l’IA arrive à créer des îlots cohérents : des produits proches visuellement se regroupent
- Cela valide que l’image contient suffisamment d’information pour aider à choisir une catégorie automatiquement
- Concrètement ça sert à : pré-catégoriser un nouveau produit, repérer des erreurs de catégorie, accélérer la mise en ligne

```
Label majoritaire par cluster :
cluster
0      Beauty and Personal Care
1                 Home Furnishing
2               Kitchen & Dining
3                     Baby Care
4                        Watches
5    Home Decor & Festive Needs
6                      Computers
Name: label_lvl1, dtype: object
```



Si les groupes sont séparés, ça veut dire qu’on peut automatiser une grosse partie de la catégorisation à partir ds photos.

Made with GAMMA

Passer des “mots” au “sens”

Objectif

Regrouper les produits via leur description.

- Approche testées : CountV / TF-IDF / SBERT (ici contrairement aux autres, SBERT essaie de capter ce que la phrase veut dire.

Résultats

- Count → ARI 0.134 / NMI 0.301
- TF-IDF → ARI 0.284 / NMI 0.484
- SBERT → ARI 0.405 / NMI 0.566 ✅

Interprétation

Quand on comprend le “sens” des descriptions plutôt que de compter les mots, on regroupe mieux les catégories… mais l’image CNN reste encore plus discriminante.

Made with GAMMA

Regroupement par “sens” avec SBERT (2D)

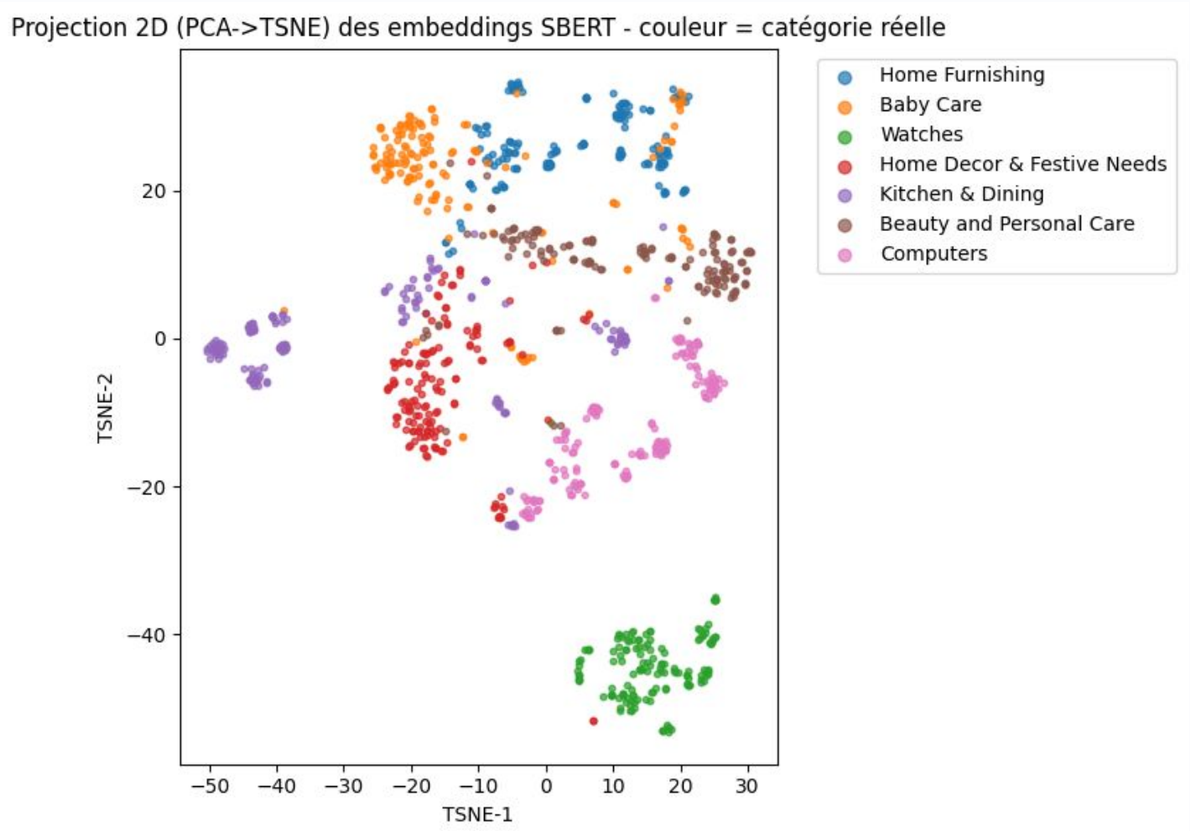
Ce que montre le graphe

- On voit des zones assez compactes pour certaines catégories -> les descriptions se ressemblent vraiment
- Exemple visible : Watches forme un gros groupe très net (en bas) -> facile à regrouper avec le texte
- D’autres catégories sont plus mélangées -> leurs descriptions utilisent des mots proches (“set”, “pack”, “new”, “material”...) donc le texte seul a du mal à trancher.

Pourquoi c’est utile pour notre projet

- Cela valide que le texte peut aider à regrouper des produits, mais pas parfaitement
- Cela sert surtout à repérer les catégories faciles (qui se regroupent naturellement et les catégories ambiguës (qui se complète avec l’image)

Là, on ne compare plus les mots : on compare l’idée de la description. Quand les points d’une couleur se rassemblent, ça veut dire que la catégorie est reconnaissable juste au texte. Quand ça se mélange, c’est qu’il faut l’image pour trancher.



Made with GAMMA

Synthèse des approches (texte vs image) & reco

	Famille	Représentation	Clustering	ARI	NMI
6	Image	ResNet50 TL	KMeans sur t-SNE 2D	0.521000	0.592000
5	Image	ResNet50 TL	KMeans sur features	0.492857	0.587465
4	Image	ORB BoVW	KMeans sur features	0.012588	0.030569
3	Texte	TF-IDF	KMeans sur t-SNE 2D	0.483000	0.579000
2	Texte	SBERT	KMeans sur features	0.405360	0.566403
1	Texte	TF-IDF	KMeans sur features	0.284000	0.484000
0	Texte	CountVectorizer	KMeans sur features	0.134000	0.301000

Points clés à conclure :
1) Texte : TF-IDF > Count ; SBERT améliore nettement TF-IDF en ARI.
2) Texte : KMeans sur t-SNE 2D peut augmenter les scores, mais t-SNE sert surtout à visualiser.
3) Images : ORB-BoVW ne fonctionne pas sur ce dataset (scores quasi nuls + mélange visuel).
4) Images : Transfer Learning CNN (ResNet50) donne la meilleure séparation et confirme la faisabilité.
5) Recommandation : privilégier CNN, et envisager un supervisé pour les catégories proches.

Made with GAMMA

Classification supervisée des produits à partir des images

Objectif

- Prédire automatiquement la catégorie d'un produit à partir de sa photo
- Montrer qu'un modèle "apprend" réellement.

Méthode

- Transfer Learning : on utilise un modèle déjà entraîné ailleurs pour gagner du temps et des performances
- MobileNetV2 : modèle "léger" et rapide, pré-entraîner sur ImageNet (grande base d'images)
- DataAugmentation : on crée des variantes d'images (retournement, rotation, zoom, contraste) pour rendre le modèle plus robuste)
- Split/train/validation : 80% pour apprendre, 20% pour tester

Résultat :

Meilleure précision validation : environ 82%

Le modèle progresse vite : on passe d'un démarrage faible à un niveau stable en quelques époques.

```
Epoch 1/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 78s 3s/step - accuracy: 0.3000 - loss: 1.9046 - val_accuracy: 0.6524 - val_loss: 1.0363
Epoch 2/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 60s 2s/step - accuracy: 0.7196 - loss: 0.8927 - val_accuracy: 0.7667 - val_loss: 0.7533
Epoch 3/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 81s 2s/step - accuracy: 0.8172 - loss: 0.6330 - val_accuracy: 0.7810 - val_loss: 0.6656
Epoch 4/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 82s 2s/step - accuracy: 0.8308 - loss: 0.5235 - val_accuracy: 0.7952 - val_loss: 0.6417
Epoch 5/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 59s 2s/step - accuracy: 0.8707 - loss: 0.4498 - val_accuracy: 0.8190 - val_loss: 0.6000
Epoch 6/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 59s 2s/step - accuracy: 0.8879 - loss: 0.3740 - val_accuracy: 0.8095 - val_loss: 0.6299
Epoch 7/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 71s 3s/step - accuracy: 0.8718 - loss: 0.3710 - val_accuracy: 0.8095 - val_loss: 0.5852
Epoch 8/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 85s 3s/step - accuracy: 0.8708 - loss: 0.3576 - val_accuracy: 0.8238 - val_loss: 0.5901
Epoch 9/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 60s 2s/step - accuracy: 0.9043 - loss: 0.2970 - val_accuracy: 0.8143 - val_loss: 0.5505
Epoch 10/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 70s 3s/step - accuracy: 0.9117 - loss: 0.2945 - val_accuracy: 0.8190 - val_loss: 0.5571

Meilleure val_accuracy: 0.8238
Meilleure val_loss: 0.5505
```

Ici on ne regroupe plus, on prédit. On part d'un modèle déjà très fort (MobileNetV2 entraîné sur ImageNet), on lui apprend nos 7 catégories, et on ajoute de la data augmentation pour qu'il soit robuste aux photos différentes. Résultat : ~82% de précision sur des images jamais vues, donc la catégorisation automatique est réaliste.

Made with GAMMA

Performance du modèle - Matrice de confusion

Ce que montre le graphe

- Accuracy validation : environ 82% sur 210 images
- La matrice de confusion est “bonne” quand la diagonale est claire : ici on a une diagonale bien visible, donc le modèle distingue plusieurs catégories.

Baby Care : precision 0.769 | recall 0.667 | F1 0.714

Beauty & Personal Care : 1.000 | 0.700 | F1 0.824

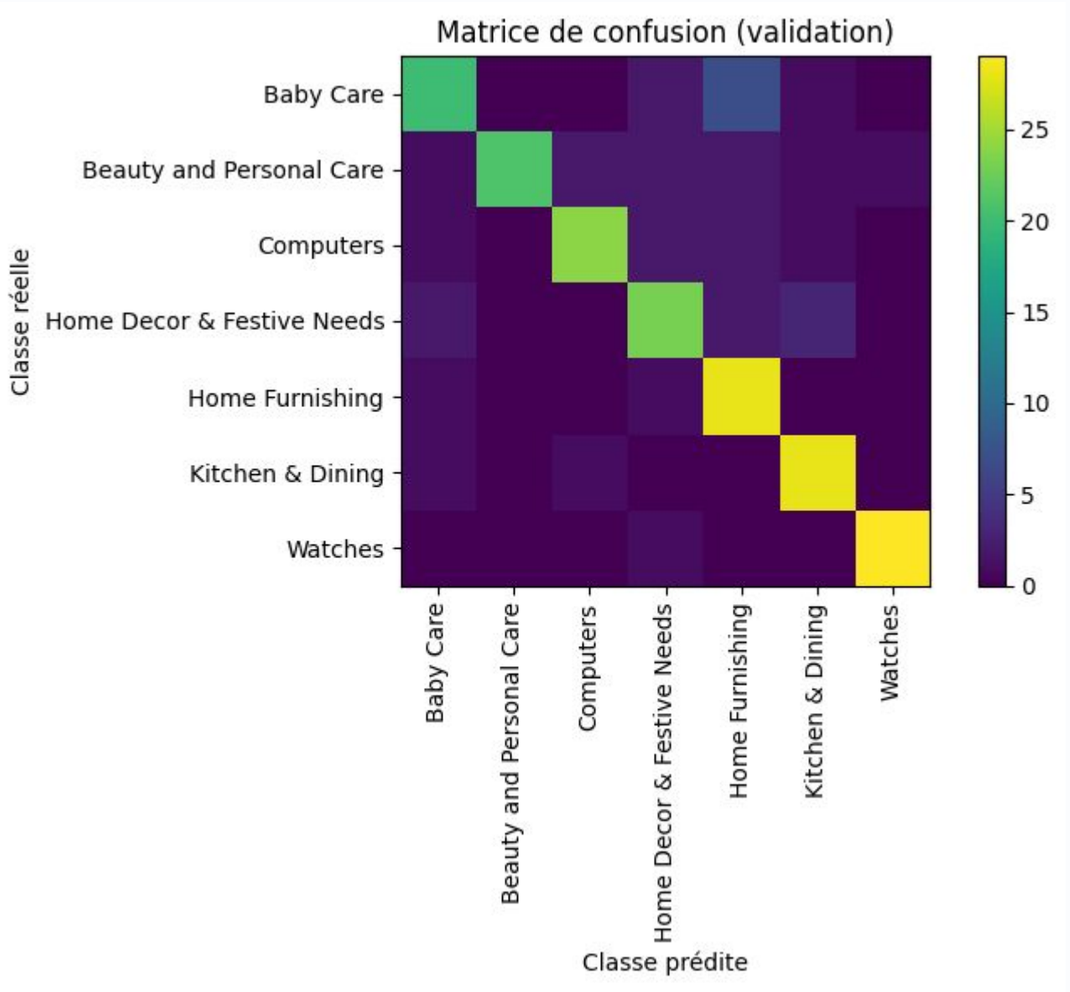
Computers : 0.889 | 0.800 | F1 0.842

Home Decor & Festive Needs : 0.742 | 0.767 | F1 0.754

Home Furnishing : 0.683 | 0.933 | F1 0.789

Kitchen & Dining : 0.824 | 0.933 | F1 0.875

Watches : 0.967 | 0.967 | F1 0.967



Made with GAMMA

Exemples de prédictions - quand le modèle a raison

- Chaque image = 1 produit du jeu de validation (données jamais vues pendant l'entrainement)
- "Vrai" = catégorie réelle, "Prédit" = catégorie choisie par le modèle.
- p = niveau de confiance : plus c'est proche de 1, plus le modèle est sûr.
- Ici, on voit qu'il réussit surtout sur des objets très "typés" visuellement (montres, mugs, déco)

Même quand il a raison, le modèle peut hésiter si l'image est ambiguë : la prédiction est bonne, mais p est plus bas.



API OpenFoodFacts : génération d'un dataset test

foodId	label	category	foodContentsLab	image	local_image_path
8001841769189		Champagne		https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/8001841769189.jpg
3113934004147	Canard Duchêne	Champagne	Pinots et de Cha	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3113934004147.jpg
3258431220000		Champagne	Champagne	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3258431220000.jpg
3114080034057	Champagne rosé	Champagne		https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3114080034057.jpg
3185370729960	Br MOË HANDC	Champagne	IMPERIAL LOOI	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3185370729960.jpg
3282946015837	Nicolas Feuillatte	Champagne	Champagne, Co	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3282946015837.jpg
3049610004104	Veuve Clicquot (	Champagne	Champagne	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3049610004104.jpg
20712907	Champagne	Champagne		https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/20712907.jpg
3113910312013	Champagne Alfr	Champagne	Champagne bru	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3113910312013.jpg
3185370283905	Champagne Rui	Champagne	champagne	https://images.openfoodfacts.o	/content/drive/MyDrive/Master AI Engineer/Colab Notebooks/Projet 6/openfoodfacts_images_champagne/3185370283905.jpg





Objectif

- Récupérer des produits réels et fabriquer un fichier exploitable pour tester une IA de catégorisation.

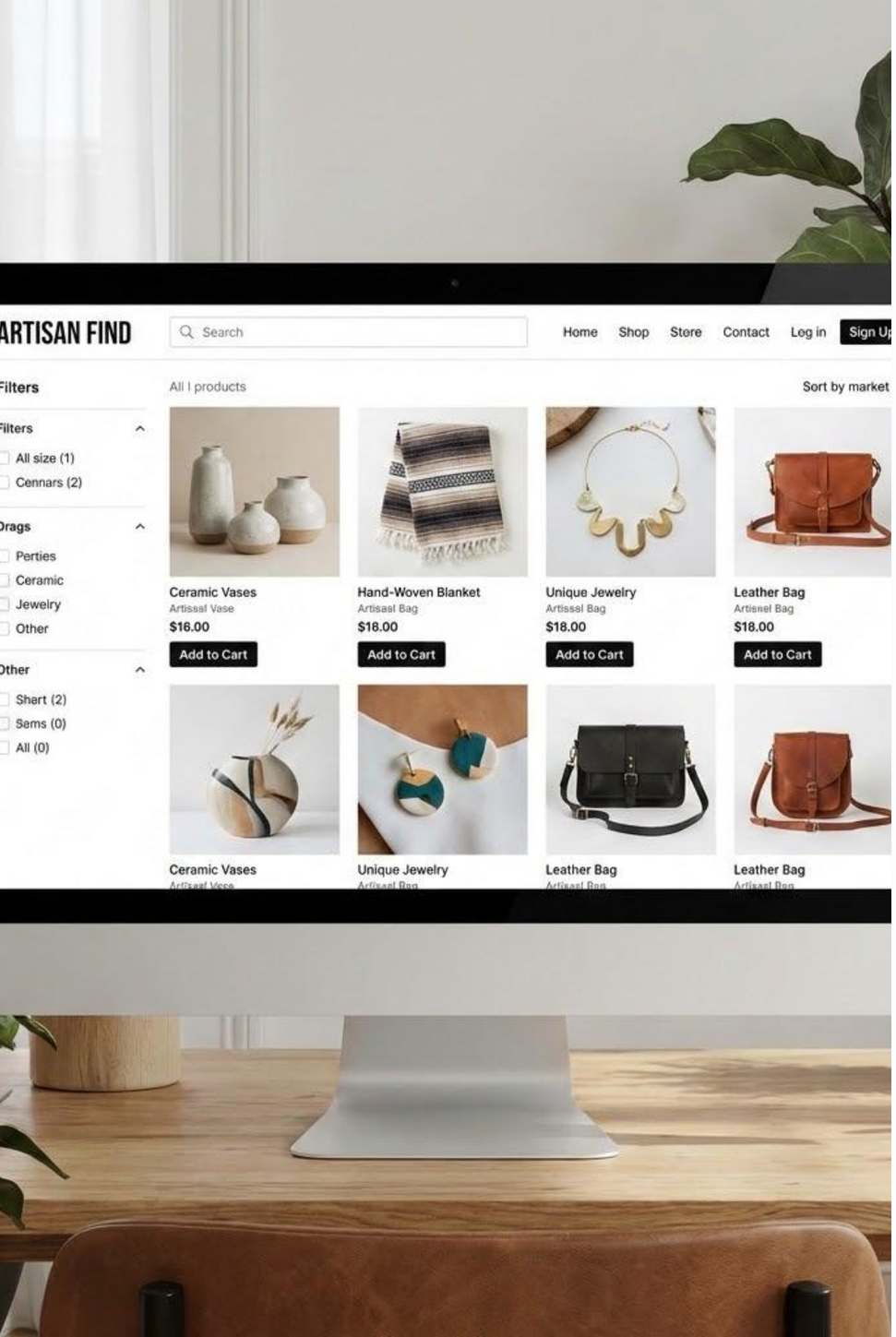
Ce que le script fait :

- Récupère des produits via l'API OpenFoodFacts (tag "champagne")
- Nettoie : champs essentiels, suppression doublons, catégorie harmonisée
- Télécharge les images et ajoute un chemin local dans le CSV

Ce que j'obtiens :

- Un CSV "clean" avec : id produit + nom + catégorie + texte + image + chemin local
- Un dossier d'images local réutilisable pour entraîner/tester un modèle

Made with GAMMA



Conclusion et Recommandations

Ce qu’on a prouvé

- On peut regrouper / classer des produits en 7 catégories à partir des données fournies.
- Les images sont le signal le plus fiable : elles décrivent mieux la catégorie que le texte court.

Limites observées

- Certaines catégories se ressemblent visuellement (ex : “Home Furnishing” vs “Home Decor…”)
- Le texte seul aide, mais il est trop court / bruité pour être aussi fiable que l’image.

Recommandations

Fusionner les images et le texte pour mieux séparer les catégories proches, augmenter les données / équilibrer les classes (plus d’exemples), déploiement d’une prédiction automatique + contrôle humain sur les cas “incertain” (alerte automatique)

Made with GAMMA